Unified Agent Benchmark v0.1: Specification, Coverage, and What Is Bound

A Frozen Contract for Certifying Persistent, Adaptive and Self-Improving Agents Across Six Domains, with a Generated Coverage Matrix

Chengshuai Yang[1,*]

[1]NextGen PlatformAI C Corp, USA

[*]Correspondence: spiritai@platformai.org

September 3, 2026 — draft v0.1

Abstract

A benchmark built after the agents it scores is a benchmark shaped by what those agents happen to do well. This paper freezes the Unified Agent Benchmark (UAB) before the agents it is meant to certify are optimized. UAB v0.1 is a measurement contract for Unified levels U0–U3 across six domains — paper development, funding, job search, software engineering, research and business workflows — built on the six-coordinate framework of Yang and Xue [3]. The contract fixes the unit (a frozen model–harness pair), the primitive (delivered outcomes net of false completions, with a class loss ratio ρ that sets the reliability a class requires), the record every episode must carry (band, budget, reliability, resource envelope, verifier version, cost, and why the episode ended), the admissible acceptance loci (never the performing system, never the same process in a reviewer's persona), the task-manifest schema and freeze-by-hash rule, and four policies for intervention, resources, verification and retention. It then reports coverage honestly and by generation rather than by hand: of 36 domain-band cells, 4 are bound, 7 are generator-bound, 2 have something running at the band for another family, and 23 are specification only; 10 cross-domain coordinate suites — four restart tests for $M1$, three grounded self-state tests, and three delegation starters — are bound. Every bound cell names a public asset and validates against the schema. Six families — one explicit code edit, one funding requirement, the hard requirements of a job posting, one citation verified against its reference, a routine results section written from supplied evidence, and one company fact — are built inside the package with seeded generators, hidden keys, a reference solver, the plausible wrong method and a specification–key test; each discriminates on every one of 48 seeds, and a live check with a real executor passed all six after exposing one specification defect, which was repaired. Every domain now has at least one runnable cell. A U2 learning-transfer suite is bound as a paired-episode family: a convention learned from verified feedback in one episode must decide the answer in a different task after a process restart, scored against the same pair with its memory ablated; a live protocol run with a real executor is reported, together with the two defects it found in the suite's first version. What v0.1 can measure is a U0–U1 delegation reading in the software domain, a T0 reading in the other five domains and a T1 reading in paper development, $M1$, an $I2$ learning-transfer reading, $SA1$–$SA2$ and $C0$–$C1$; what it cannot yet measure is any U3 claim, because no self-improvement suite is bound, and anything above T0 in funding, job search or business. The paper states the binding order that would change that, and the validity requirements — specification–key tests, concordance audits, graded scoring, headroom, termination reasons — that every future binding must meet, because the benchmark this one replaces was well formed on every row and could not draw a frontier.

Keywords: agent benchmark; Unified Intelligence; delegation; false completion; restart test; verifier separation; coverage matrix; pre-registration

Contents

1 Introduction

The dangerous sequence is: build an agent, observe what it can do, design a benchmark around those capabilities, and watch the agent pass. The sequence this paper commits to is the reverse: define the intelligence claim, construct the benchmark, freeze the verifier, build the agent, measure. Formally, a frozen benchmark B precedes the sequence of candidate pairs $A_0, A_1, A_2, \ldots$ and its acceptance criteria stay outside every candidate's write set [2].

This is not a new principle, and the reason to write it down again is that the corpus this paper belongs to has already violated it once by accident. DLI-Bench v0.1 was well formed on every one of its 96 rows and could not produce a delegation frontier, because the intervention budget, the reliability target, the intervention-depth ceiling, the time budget and four of eight difficulty coordinates each took exactly one value per task band: one independent variable wearing six names [1]. Nothing in the rows was wrong. The set of them measured

less than it looked like it measured. That failure is the design constraint here: coverage is generated from manifests, never asserted in prose, and a cell is counted only when a runnable asset exists for that domain and that band.

1.1 Contributions

1. **A frozen measurement contract** (§2): unit, primitive, four outcomes, the per-episode record, and the gates for U0–U3.
2. **A task-manifest schema** (§3) with freeze-by-hash, an admissible-locus ladder for acceptance, and binding status as a required field.
3. **Six domain band ladders** T0–T6 (§4), with the rule that T5 and above admit no static item.
4. **Four policies** (§5): intervention, resources, verification, retention.
5. **A generated coverage matrix** (§6): 36 domain-band cells labelled from manifests, 24 bound manifests with public assets, eight in-package families proven to discriminate, and a validator that fails on any inconsistency.
6. **What v0.1 can and cannot measure** (§7), and the binding order for v0.2 (§9).

1.2 What this paper does not claim

No agent has been certified on UAB v0.1. No score appears here beyond a six-item live check. The bound cells are a small core — every domain at T0, paper at T1, software through T4, one sealed research family — plus cross-domain coordinate suites. Those facts are the subject of §6, not a footnote to it.

2 The Measurement Contract

2.1 Unit and profile

The unit is a frozen model–harness pair $A(m,h)$. The output of a certification run is a profile, not a score:

$$[C, I, O, T, H, SA] \text{ at reliability } p, \qquad M \text{ reported beside } I,$$

with U^* the highest Unified level whose gate and every lower retention suite pass. For an individual unit, O is omitted from the gate and named as omitted; it is never set to zero, and `unmeasured` is never `0`.

Level	Gate (all must hold)	Delegation gate
U0 Reactive	C0, I0, O0, SA0	T0 at H5–H4
U1 Persistent	C1, I1 (requires $M1$), O1, SA1	T1 at $\leq$H3
U2 Adaptive	C2, I2 (requires $M3$), O2, SA2	T2 at $\leq$H2
U3 Self-improving	C3, I3 (requires $M4$), O3, SA3	T3 at $\leq$H1

Table 1: The U0–U3 gates in scope for UAB v0.1, from Yang and Xue [3]. Retention: $U_n \Rightarrow U_{n-1}$.

2.2 The primitive and the four outcomes

Every delegation episode records two fields separately: `harness_accepted` (null where the harness has no acceptance step) and `verifier_pass`. Four outcomes are derived from them and never typed by hand:

delivered and correct; *false completion* (handed back as done, and wrong); held back (a correct refusal, which costs human load and not reliability); false rejection (the cost of strictness). The scoring primitive is

$$S_A^{\text{net}}(T,H) = P(\text{delivered and verifier pass}) - \rho(\tau)\,P(\text{false completion}), \quad \text{clamped at } 0,$$

where $\rho(\tau)$ is the class's loss ratio and $p^* = \rho/(1+\rho)$ is the reliability the class requires. Setting $\rho = 0$ reproduces the success-only figure for audit. A class with unbounded residual harm has $p^* = 1$ and is in the benchmark to be refused; a suite without such classes measures only the classes where attempting is safe.

Why the first term says "delivered"

A success-only aggregate is exactly invariant to an acceptance step: it changes which results are reported as successes, not whether they are correct. Measured on 48 episodes with one frozen model and only the harness changing, two rungs differing only by acceptance scored identically while false completions went from 2/12 to 0/12. And a primitive whose first term counted verifier passes regardless of delivery would be unchanged by any acceptance policy, so a harness that held back everything would score best. The delivered condition closes both.

2.3 What every episode records

The tuple (T,H,p,R,V,C): band, budget, verified reliability, resource envelope, verifier version, cost. Per episode: attempts, harness iterations, model calls, wall time, tokens, exit status, whether the limit was reached, a termination reason in $\{\texttt{normal},\texttt{timed\_out},\texttt{crashed},\texttt{not\_attempted}\}$, every intervention with its depth, and a forecast identifier where a self-awareness probe is attached. A run the runner killed is recorded as timed out or not attempted, never as a delivery: a truncated run and a wrong answer are the same row otherwise, and the framework this benchmark serves had to retract a headline number for exactly that reason.

For U1 and above: restart success, provenance recovery, stale-state suppression. For U2 and above: held-out transfer, repeat-error reduction, lesson rollback. For U3 and above: self-improvement gain and regression rate. Always: false-completion rate, held-back rate, false-rejection rate, human load, cost per verified completion, verifier disagreement, and per-band headroom.

3 The Task Manifest

Every task family is one JSON manifest validating against a published JSON Schema (draft 2020-12). A manifest is hashed with its hash field empty and frozen by that hash; a change is a new version. The required blocks:

Block	Content
identity	`task_id` (`uab-<domain>-t<band>-<family>`), benchmark version, domain, family, target coordinates
difficulty	the band, and optionally the difficulty vector (horizon, uncertainty, ambiguity, verification, novelty, environment change, tool diversity, coordination, risk)
human intervention	the budgets the family is scored under, and the intervention-depth ceiling above which an episode is void at that budget
mission	description, deliverables, and whether every rule the key grades is stated in the visible specification
resources	the wall-clock limit *the runner enforces*, model calls, tokens, web and file access
authority	what the executor may do; anything absent is forbidden; outward actions default to false
acceptance	the weakest admissible locus, whether a criterion register is required (always), the bound on σ (the share of criteria the system may author), the required success rate, ρ, and hard failures
verifier	type, version, hidden or public, whether a specification–key test ships, public reference
retention	the lower-level suites that must pass
binding	`BOUND`, `GENERATOR_BOUND`, `BAND_ONLY` or `SPECIFICATION_ONLY`, the asset, a note
provenance	frozen date, source, content hash

Table 2: The manifest blocks. The schema rejects a manifest with an inadmissible acceptance locus, a bound status without an asset, or an unknown field.

The acceptance-locus ladder

a0, the performing system: never admissible. a1, the same process wearing a reviewer persona: never admissible. a2, a separate process on a *copy* of the deliverables with no inherited environment: the weakest admissible locus. a3, a separate host. a4, a separate party that holds the ground truth and returns a scalar. a5, cryptographic promotion. One base model in two processes with separate credentials and enforced write sets satisfies a2; one model told to now act as reviewer does not. The requirement is on locus, not on species.

An abridged bound manifest, for the software T1 family:

```
task_id: uab-code-t1-request_timeout     domain: code   family: request_timeout
target_coordinate: [delegation]          difficulty.T_band: T1
human_intervention: budgets [H0,H1,H2], max_cid 1
mission: change the request timeout to the value the goal names and leave
         the cache and retry constants unchanged;   hazard_disclosed: true
resources.wall_time_seconds: 600 (enforced)
authority: write_workspace, run_commands (network, send: false)
acceptance: locus a2_separated_process, register required, sigma_bound 0.0,
            required_success_rate 0.80, rho 1.0,
            hard_failures [benchmark_tampering, criterion_edit_after_seal,
                           delivery_of_killed_run]
verifier: deterministic_check, dli-ladder/HG0-HG3@2026-08-25, hidden,
          spec_key_test: true
binding: BOUND, asset .../delegation/ladder.py::t1.request_timeout
         note: two checks -- the named value changed; the neighbours did
         not move
provenance: frozen 2026-09-03, hash sha256:...
```

The second check is the whole item: a global search-and-replace passes the first and fails the second.

4 Bands by Domain

Band	Paper	Funding	Job	Code	Research	Business
T0	one citation, table or figure generated or verified	one deadline or eligibility requirement extracted	requirements extracted from one posting	one explicit code edit	one fact retrieved or calculated	one market or company fact
T1	routine section from supplied evidence	one call evaluated against one applicant	fit assessed for one position	routine bug fixed with tests	one known analysis reproduced	one bounded analysis
T2	multi-step literature, analysis, figure and writing task	find, compare and rank opportunities	search, compare, tailor, keep state across applications	multi-file feature	specified multi-step workflow	multi-step competitor, customer or market assessment
T3	complex revision with verification	funding strategy and proposal workflow	multi-company application strategy	ambiguous repository-scale failure diagnosed and recovered	strategy chosen, failures diagnosed, alternatives run	strategy under uncertainty and conflicting evidence
T4	full expert paper project under a frozen question	long-horizon grant preparation	complete long-horizon campaign	long-horizon software project	complete project under a frozen benchmark	long-running business project
T5	paper requiring an unknown method	non-obvious strategy discovered and validated	strategy that improves verified outcomes	unknown solution method for a sealed task	unknown mechanism discovered and validated	new strategy or opportunity validated
T6	mission: project, experiments, manuscript, submission	mission: funding portfolio	mission: full campaign	mission: projects	mission: project portfolio	mission: project portfolio

Table 3: Difficulty bands per domain, from Yang [2] §10. Bands are cumulative in what the human still supplies, not in domain knowledge.

T5 and above admit no static item

A level defined by the answer not being known in advance cannot be certified by an item whose key is recorded before the run. The only items that reach past the mechanically-derivable envelope are generated after the pair is frozen and graded on extrapolation [3]. The one T5 cell bound in v0.1 is of that kind: a sealed mechanism generated per seed, scored on held-out points outside the observation box against a nearest-neighbour baseline.

5 Policies

5.1 Intervention

Budgets H0 (none), H1 (exception-only: one unavailable fact or one narrow clarification, no strategy), H2 (occasional: at most two bounded corrections, no central strategy), H3 periodic, H4 frequent, H5 continuous. Every intervention is logged with its Critical Intervention Depth: CID0 permission, CID1 external fact, CID2 local correction, CID3 strategy, CID4 decomposition, CID5 central design, CID6 the human did the task. Governance approvals are logged in a separate table and never count as cognitive intervention unless the approval supplies task strategy. An intervention deeper than the manifest's ceiling voids the episode at that budget. **The zero-intervention rule:** an H1 or H2 run that raised zero interventions is reported as conducted at H0; cells that differ only by label are one cell. Human load — count, minutes, authorization latency — is reported beside the score.

5.2 Resources

Every pair report carries the envelope R: wall clock, model calls, tokens, tool calls, subagent count, compute class, memory budget, external-knowledge access. The wall-clock limit is enforced on the executor's process group; a limit stated and not enforced, or one that kills only the direct child, is an apparatus defect. Default ceilings by band: 300, 600, 1800, 3600, 7200 seconds for T0 to T4. Cost per verified completion is a primary endpoint. Tool authority is equalised across executors or recorded as unequal.

5.3 Verification

The criterion register is written before the work, refuses a criterion about an existing deliverable, refuses a duplicate, refuses a criterion that does not say what it misses, seals when execution starts, and is hash-chained so an edit is detectable. It reports σ. Public: the verifier type, the specification, worked examples, the reference implementation. Hidden: the scored instances or key. A hidden key is an assertion until something tests it: parameterised families ship a specification–key test run on drawn instances against the key's own reference implementation, and fixed items are concordance-audited when two executors of materially different capability fail with the same response. The verifier version is part of every result. Verifier disagreement above ten percent withdraws an item until audited. Families with many sub-checks report a fraction and separate failure modes.

5.4 Retention

Promotion to U_n requires the U_n suite and every lower retention suite to pass in the same frozen configuration in the same run. A candidate that raises a higher score while lowering a lower suite below its gate is not promoted, and the regression is a primary result. Longitudinal levels (I3+, O3+, SA4+, U4+) require several cycles. Two rungs that differ only in post-hoc machinery are scored from the same executor outputs.

5.5 Separation

Protected, outside every candidate's write set: hidden tests, frozen manifests, thresholds, held-out data, the primary verifier, certification task selection, the promotion decision. Writable: agent code, prompts, memory policies, planning policies, routing, workflow, sub-agent organization, candidate improvements. The designer may design. It may not grade itself.

6 Coverage

Coverage is generated by a script from the manifests and validated by a second script that fails on any inconsistency: a manifest that does not match the schema, a hash that does not match its content, a bound status without an asset, a matrix cell naming a manifest of another domain or band, or a local asset path that does not exist. A cell is `BOUND` only when a manifest exists for that domain and that band; `BAND_ONLY` means something runs at the band for another family, and is never counted.

Domain	T0	T1	T2	T3	T4	T5
paper	**GEN**	**GEN**	spec	spec	spec	spec
funding	**GEN**	spec	spec	spec	spec	spec
job	**GEN**	spec	spec	spec	spec	spec
code	**GEN**	**BOUND**	**BOUND**	**BOUND**	**GEN**	spec
research	band-only	band-only	spec	spec	spec	**BOUND**
business	**GEN**	spec	spec	spec	spec	spec

Table 4: The UAB v0.1 coverage matrix as generated. 36 cells: 4 bound, 7 generator-bound, 2 band-only, 23 specification-only.

6.1 The bound cells and their assets

Cell	Family	Verifier	Asset
code T0	one edit	every other file byte-identical to a snapshot; exactly one line of the target differs; the named value set	`families/code_t0.py` in this package
funding T0	extract requirement	normalized exact match against the generator's key; value must appear in the source; the target is never the first date, eligibility line or USD figure	`families/funding_t0.py`
job T0	extract requirements	set equality on the required bullets (graded by Jaccard), two flags, location	`families/job_t0.py`
paper T0	verify citation	exact discrepancy set over first author, year, venue, pages, plus the consistent flag; the target always carries a non-year discrepancy	`families/paper_t0.py`
paper T1	results section	seven prose checks: best method by the stated direction, its exact value, runner-up, margin, table reference, dataset, and no decimal number absent from the evidence	`families/paper_t1.py`
business T0	extract fact	exact value and unit, and a source line that appears verbatim and contains the value; a peer and a restated figure precede the target	`families/business_t0.py`
code T1	request timeout	two deterministic checks: the named value changed; the neighbours did not	`delegation/ladder.py` in AI4Science
code T2	pipeline	the rejected count in the report equals the unusable rows in the input	same
code T3	search latency	timing criterion registered before the work	same
code T4	expression language; shift cover	exact reference (about 238 expressions per seed); exact optimum (60 instances per seed); computed keys, spec–key test on every draw	`dli_bench` generators in AI4Science
research T5	sealed mechanism discovery	held-out extrapolation RMSE at or below 25% of nearest-neighbour baseline; mechanism statement required	regime-switch generator, protocol in the v2.0 paper dataset
coordinate I1 $\times 4$	restart continuity, temporal order, relevant retrieval, provenance	restart probe after a genuine process restart	HIL library v1.1 starters
coordinate SA1/SA2 $\times 3$	identity role, tool reachability, authority limits	grounded self-state against stale textual cues	same
coordinate DI T0/T1 $\times 3$	data analysis (T0, T1), document workflow (T0)	independent reference, H0–H5 matrix	same

Table 5: Every bound manifest names a public asset. The coordinate suites are cross-domain and are not counted as domain coverage; their starter instances are development examples, and hidden instances are required for formal certification.

6.2 The six in-package families, and the live check

Six families — code T0, funding T0, job T0, paper T0, paper T1 and business T0 — are built inside the benchmark package with no dependencies. Each ships a seeded generator with a hidden key, a verifier, a reference solver that never sees the key, the plausible wrong method the family exists to catch (a global search-and-replace; first-match extraction; the first bulleted list; comparing the year only; the largest number on the first dataset; the first figure after the keyword), and a specification–key test that also asserts the trap fired on the instance. The self-test requires the reference solver to pass and the naive method to fail on *every* seed, because a generator whose trap is probabilistic fills the suite with instances that discriminate nothing; the first version of two families fired the trap on only half the seeds and was repaired so that the target is never the first date, the first eligibility line, the first USD figure or the first list. Result: reference 48/48, naive 0/48, specification–key agreement on 100 seeds, for all six.

A gate that has only ever been opened by its own reference solver has not been shown to open. One seed of each family was run through a real executor (Claude Code 2.1.258, headless, file edits only, four to five minutes, killed by process group). It passed five items outright and failed the job item on one check: asked for "the Location line, verbatim", it returned the line including its `Location:` label. That is a specification defect of exactly the shape the framework's concordance rule exists for — the prose did not state the rule the key graded — and it was repaired in the goal text and the verifier; the rerun passed. The three families written after that repair passed on the first run. Six items, one executor, one seed each: a reading that the gates open, not a rate. The results-section item is worth one remark: the executor's paragraph named the two trailing methods as well as the best and second-best, and the fabrication check passed because it stated no number for them; a section that had quoted their values would also have passed, since those values are in the evidence, and one that had rounded them differently would not.

6.3 The U2 learning-transfer suite

An $I2$ claim requires that verified experience at time t improves held-out behaviour at $t+1$, with $M3$ beneath it: the lesson must survive a restart and a rebuilt retrieval index. No single-episode item can test that, so the suite is a *pair* of episodes per seed sharing one organization convention the task text never states — a repeated id is a correction and the last row wins; dates are day-first; amounts are in cents; blank-name rows are void; per-id output is ordered descending. Both episodes disclose the baseline conventions and say that exactly one may be overridden by a convention the organization has stated in previous work, without saying which.

Episode A asks the pair to clean a ledger. Under the baseline reading its first attempt fails one hidden registered check; the harness returns a feedback file naming the convention and stating that it applies to all future work for this organization; a second attempt should pass. The process is then terminated and episode A's workspace is removed. *Episode B* is a different task — per-id and per-month totals from a different ledger — in a new directory, where the same convention silently decides the answer. The score is the paired difference $\text{pass}(B \mid \text{after } A) - \text{pass}(B \mid \text{memory ablated})$; the floor is the ablated arm, the same pair with a fresh store on episode B alone. By construction the baseline reading fails B on every seed, and the specification–key test asserts it per instance. Self-test: a solver carrying the lesson passes A's second attempt and B on 48/48 seeds; the baseline solver fails B on 48/48 with the failure mode `default_reading_no_transfer`.

What the suite does and does not certify

It supplies the $I2$ evidence: a lesson from one episode changed behaviour in a later, different one, after a restart, and the ablation shows the lesson did the work. It does not by itself certify $M3$: the runner exposes a hook to rebuild the pair's retrieval index between episodes and does not perform the rebuild for an arbitrary executor. And a U2 gate still requires $C2$, $SA2$, T2 at $\leq$H2 and retention of U0–U1, which are separate suites.

The live protocol run. The protocol was run against one real pair — Claude Code 2.1.258 in headless mode, file edits only, each invocation in its own process group under a seven-minute limit — on two seeds with two different conventions (cents; void blank-name rows). The first run of the first seed found two defects in the suite, not in the pair, and both are recorded because each produced a plausible wrong reading. The task text had not disclosed the baseline conventions, so the executor's day-first reading of ambiguous dates in episode A was defensible, and the feedback about cents could not correct a failure it did not name; the second attempt failed for that reason. And the ablated arm, run after the experienced arm under a sibling directory, listed its neighbour and reported seeing the other arm's answer, which it

said it did not read; a protocol that relies on that sentence has no control arm. The baselines are now disclosed in both episodes, the ablated arm runs first under a separate root, and episode A is archived outside the project before B starts.

Under the repaired protocol both seeds read *transfer*: attempt 1 failed under the baseline, attempt 2 passed after feedback, episode B passed after the restart, and the ablated arm failed B with the failure mode `default_reading_no_transfer`, so the paired difference was 1 on both. Two more facts about the pair belong in the record. The executor tried in every episode to write the convention to its persistent memory store and was refused, because the store is outside the project and the tool grant did not allow it; the lesson carried anyway, through the per-project session records the harness keeps and the executor read back. That is the harness supplying persistence the executor was denied, which is what the suite exists to detect. And the executor in episode B also announced which baseline it was keeping, correctly, on both seeds. Two seeds and one pair are a reading that the suite's gate opens for a pair with persistence and closes for the same pair without it; they are not a rate, and they certify neither $I2$ nor $M3$ for that pair, since the retrieval-index rebuild was not exercised.

6.4 The U3 self-improvement suite

An $I3$ claim requires governed modification of a cognitive mechanism: the pair proposes a change to something it runs on, a locus it cannot write to evaluates the change on a frozen hidden set, and a promoter outside the pair decides. The suite makes the writable object concrete and small: the pair's operating policy, a `PROFILE.md` prepended to every episode — a D2 write set, policy rather than memory. Per seed, an organization has a seeded subset of three to five of the five conventions of §6.3, never stated.

Dev. Three cleaning tasks under the baseline profile; after each, the pair receives the failed checks with the convention each concerns. That is the evidence. *Propose.* The pair reads its dev evidence and writes `candidate/PROFILE.md`, general rules for every future task for this organization, or an empty `NO_CHANGE`; it never sees the hidden instances and is told that hard-coded answers and changes that break unrelated tasks will be rejected. *Evaluate*, by the locus outside the pair: baseline and candidate on five hidden instances of new data, three of them a task type not in dev (totals) so a memorised dev answer gains nothing, with the same seeds for both; and the candidate on a U0 retention item, the one-edit code task. *Promote* iff the candidate improves the hidden score by at least one instance and retention holds; otherwise reject with the reason. `NO_CHANGE` is a valid output that is never promoted.

The self-test is the promoter's discrimination, not a solver's. A scripted pair obeys `RULE` lines in its profile and nothing else; four candidates are submitted for it — the good one (the conventions as rules), an overfit one (the dev answers hard-coded), a regressing one (the good rules plus "change every occurrence", which breaks the retention item), and no change — and the promoter must promote exactly the first and reject the second and third with the right reasons. It does so on 24/24 seeds.

What one run of the suite does not certify

A promoted candidate is one improvement cycle. $I3$ and $U3$ require longitudinal evidence across several cycles, with the improvement mechanism itself versioned and rollback-able; the suite supplies the cycle and the governance, not the longitude. And the writable object here is a policy file, the shallowest of the mechanisms the framework lists; a pair that improves its retrieval or routing would need the evaluator's evidence path frozen during evaluation, which this suite does not have to do because the policy cannot touch it.

6.5 What sits behind the specification-only cells

Twenty-three cells have nothing runnable. They are not empty of design: DLI-Bench v0.2 carries 180 specification starters in six families (data 39, document 20, planning 21, research 38, software 42, tool use 20) across four strata, each with a difficulty vector, a loss ratio, an acceptance block and an intervention budget, and each marked "executable assets should be generated from the seed and kept hidden for certification" [1]. The agent-coordinate probe set carries fifteen $T{\cdot}H$ procedures per agent for the funding and job agents with benchmark-authored floors. These are where binding begins, and they are listed in the matrix as seeds precisely so that they are not mistaken for coverage.

7 What v0.1 Can and Cannot Measure

Can. A U0–U1 delegation reading in the software domain at H0, H1 and H2 on four bound or generator-bound families at T0–T3 plus two generator-bound T4 families; a T0 reading in every other domain and a T1 reading in paper development; $M1$ on four restart tests; $SA1$ and $SA2$ on three grounded self-state tests; $C0$ and $C1$ on six bound cognitive items; and a T5 sealed-discovery reading in the research domain. That is enough to certify U1 for a pair whose $C1$, $I1$ (with $M1$), $SA1$ and T1-at-H3 gates all pass, with O omitted for an individual unit.

Cannot. Any U2 claim, because no learning-transfer suite is bound: nothing here measures whether verified experience at time t improves held-out behaviour at $t+1$. Any U3 claim, because no self-improvement suite with a sandbox, a frozen hidden evaluation and an independent promoter is bound. Anything above T1 in paper development or above T0 in funding, job search or business. Any cross-domain transfer claim yet, since none of the new families has been run on the harness ladder. Any O level, since no organizational suite is bound.

The lesson carried from DLI-Bench v0.1

Every one of its 96 rows was well formed, and the set could not produce a frontier, because six fields took one value per band. The v0.1 bound core here has the same shape in miniature: three code families at one band each, at one budget in the archived runs. A frontier $F_A(H, p)$ at three budgets needs the same class held fixed across H0, H1 and H2 — the `budget_cross` stratum — and v0.1 declares those budgets in the manifests but has run only H1. The reporting profile is computable in principle and has not been computed.

8 Running a Certification

1. **Pre-register**: the pair (model identity from a live call, harness manifest hash, tool grant, memory snapshot), the manifest set and hashes, seeds, weights, ρ table, ceilings, and the intervention policy text — committed before the first episode.
2. **Apparatus checks**, each a positive control: the timeout kills a grandchild's process group; termination reasons are recorded; the gate opens on a known pass and closes on a known failure; both ends agree on versions; restore is clean; no session survives its episode.
3. **Run** the bound families at the declared budgets and seeds, restoring the memory snapshot before every delegation episode; record every field of §2.
4. **Audit**: concordance on any item two executors failed identically; per-band headroom; verifier disagreement.
5. **Score** with the delivered-outcome primitive under the predeclared weights; compute the profile, U^*, the named bottleneck, reliability with an interval, the frontier at each budget, false-completion rate, resource envelope.
6. **Report** per pair in the framework's format, with every unmeasured coordinate written as unmeasured.

9 Binding Order for v0.2

In cost order, and in the order that unblocks the most claims:

1. **The budget-cross runs** — the bound code families at H0 and H2, so the frontier profile becomes computable.

Each binding ships a specification–key test if parameterised, a concordance audit if fixed, and a public reference for the verifier type.

10 Limitations

- **The bound core is shallow.** Eleven of thirty-six cells: every domain at T0, but above T0 only software (to T4), paper development (T1) and one sealed research family. The learning-transfer suite is one family of five conventions on synthetic ledgers; a pair that learns those five has shown transfer of a stated lesson, not learning in general. The in-package families are small synthetic instances with deterministic keys; they measure whether an executor reads a specification and delivers exactly what it asked for, not domain expertise.

- **Starters are not hidden instances.** The coordinate suites' starter instances are development examples; formal certification needs withheld instances that do not yet exist.
- **No run has been scored under this contract.** The archived harness-ladder and regime-switch measurements predate the manifests and are reported in the companion paper on harness scaling, not here.
- **Band placement is by inspection.** The mapping of the three ladder families to code T1–T3, and of document and data-analysis starters to band-only cells, is a judgement recorded in the matrix source and open to revision.
- **The schema is v0.1.** It encodes the difficulty vector as optional; a v0.2 that requires it would let the collinearity check of Yang [1] run automatically.

11 Conclusion

UAB v0.1 is a frozen contract with a small bound core, and both halves of that description matter. The contract fixes what a certification is — a frozen pair, a delivered-outcome primitive with a class loss ratio, an episode record that says why it ended, an acceptance locus that is never the doer, and a hard gate with retention — before any agent is optimized against it. The core is honest about its size and its depth: four bound cells, seven generator-bound, and ten coordinate suites, with twenty-three cells named as specification only and two named as band-only so that they are not mistaken for coverage — and every domain bound at T0, which is where a T0 gate belongs and where nothing higher can yet be claimed. The next version is not a larger claim. It is the same contract with more cells bound in the order that costs least; the six cheapest — the one-edit code task, the funding and job extraction tasks the corresponding agents already need, a citation check, a results section and a company fact — were bound while this draft was written, and each was shown to discriminate before it was counted.

Availability

The specification, schema, manifests, policies, matrix and validator are in the corpus repository at `https://github.com/integritynoble/sarsi-intelligence-level` under `uab_v01/`. The bound assets are in `https://github.com/integritynoble/AI4Science` (delegation ladder and generators) and in the HIL Benchmark Library v1.1 in the corpus.

Conflicts and funding

The author reports no funding specific to this work and no competing interest.

References

[1] Chengshuai Yang. DLI-bench cannot draw its own frontier. `https://github.com/integritynoble/sarsi-intelligence-level`, 2026.

[2] Chengshuai Yang. Unified agent intelligence development plan: Benchmark-first development of general agents before AI4Science integration. Planning document, September 2026, 2026.

[3] Chengshuai Yang and Ting Xue. Measuring the unified intelligence level: A cumulative, harness-measurable hierarchy. *Version 2.3,* `https://github.com/integritynoble/sarsi-intelligence-level`, 2026.